ALFIDO TECH

Java Developer Internship

TASK 2 REPORT

Console-Based Mini Project(Student Management System)

Submitted By: Alquama Arsalan Khan

Submission Date: 22 June 2026

Objective

The objective of this task is to design and develop a console-based Student Management System using Java. This project aims to provide practical experience in implementing core Java concepts such as user input handling, loops, conditional statements, and dynamic data storage.

The system is designed to perform basic student management operations, including adding student records, viewing stored records, searching for a specific student, and exiting the application through a menu-driven interface.

This project helps in understanding how programming concepts can be applied to solve real-world problems. It also enhances logical thinking, problem-solving abilities, and programming skills by encouraging the development of an interactive application.

Furthermore, the task provides hands-on experience with Java utilities such as Scanner and ArrayList, helping in the efficient handling of user input and data management. It also strengthens the understanding of structured programming and lays the foundation for developing more advanced Java applications in the future.

Through this project, I gained practical knowledge of software development, improved my coding confidence, and learned how to create a simple yet functional management system using Java.

Project Description

Student Management System

The Student Management System is a console-based Java application developed to manage student records efficiently through a simple menu-driven interface. The application allows users to perform basic operations such as adding student names, viewing the list of stored students, searching for a specific student, and exiting the program.

The project was developed using core Java concepts including Scanner for user input, ArrayList for dynamic data storage, loops for continuous execution, and conditional statements for decision-making. These concepts work together to provide a simple and interactive user experience.

The system is designed to simulate a real-world student record management process in a simplified manner. Users can continuously interact with the application through a menu, making it easy to perform multiple operations without restarting the program.

Features of the Project

- Add new student records.
- View all stored student records.
- Search for a student by name.
- User-friendly menu-driven interface.
- Exit the application safely.

Technologies Used

- Java Programming Language

- Visual Studio Code (VS Code)
- Scanner Class
- ArrayList
- Loops and Conditional Statements

Working of the Project

The application starts by displaying a menu with different options. Based on the user's choice, the corresponding operation is performed. Student names are stored dynamically using an ArrayList, allowing records to be added and managed efficiently. The program continues to run until the user selects the exit option.

Source Code

StudentManagement.java 1

StudentManagement.java > StudentManagement > main(String[])

```java
import java.util.Scanner;
import java.util.ArrayList;

public class StudentManagement {

    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        ArrayList<String> students = new
        ArrayList<>();

        int choice;

        do {

            System.out.println("\n===== STUDENT MANAGEMENT SYSTEM =====");

            System.out.println("1. Add Student");
            System.out.println("2. View Students");
            System.out.println("3. Search Student");
            System.out.println("4. Exit");

            System.out.print("Enter Choice: ");

            choice = sc.nextInt();
            if(choice == 1){

    sc.nextLine();

    System.out.print("Enter Student Name: ");

    String name = sc.nextLine();

    students.add(name);

    System.out.println("Student Added Successfully!");
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

StudentManagement.java 1

StudentManagement.java > StudentManagement > main(String[])

```java
public class StudentManagement {
    public static void main(String[] args) {

}
if(choice == 2){

    System.out.println("\nStudent List:");

    for(String student : students){

        System.out.println(student);
    }
}
if(choice == 3){

    sc.nextLine();

    System.out.print("Enter Student Name to Search: ");

    String searchName = sc.nextLine();

    if(students.contains(searchName)){

        System.out.println("Student Found!");

    } else {

        System.out.println("Student Not Found!");
    }
}

        } while(choice != 4);

        System.out.println("Program Closed!");

    }
```

Sample Input/Output Screenshots

Screenshot 1: Adding a new Student Record

Input:

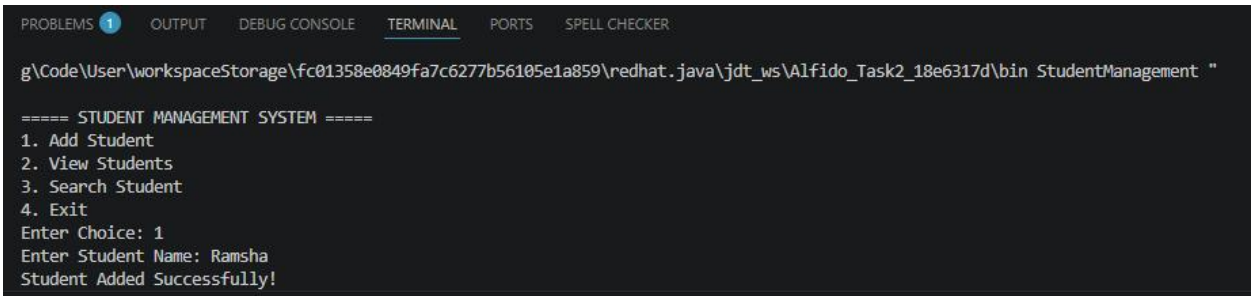
The user selects option 1 (Add Student) from the menu and enters the student's name.

Output:

The system successfully stores the entered student name and displays a confirmation message indicating that the student record has been added successfully.

Explanation:

This functionality demonstrates the use of the Scanner class for accepting user input and the ArrayList data structure for storing student records dynamically.



Screenshots 2: Viewing Student Records

Input:

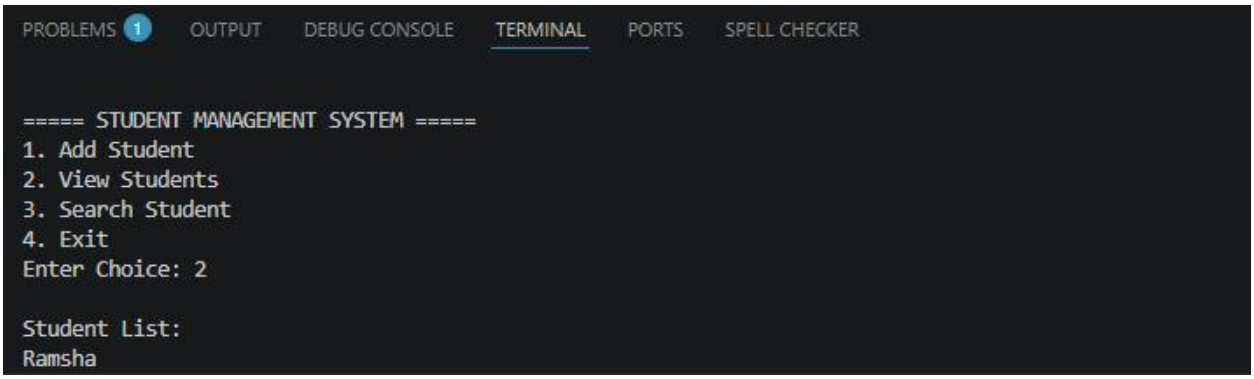
The user selects option 2 (View Students) from the menu.

Output:

The system displays the list of all student records currently stored in the application.

Explanation:

This functionality demonstrates the retrieval and display of data stored in the ArrayList using a loop. It allows users to view all available student records in an organized manner.



Screenshots 3: Searching for a Student Record

Input:

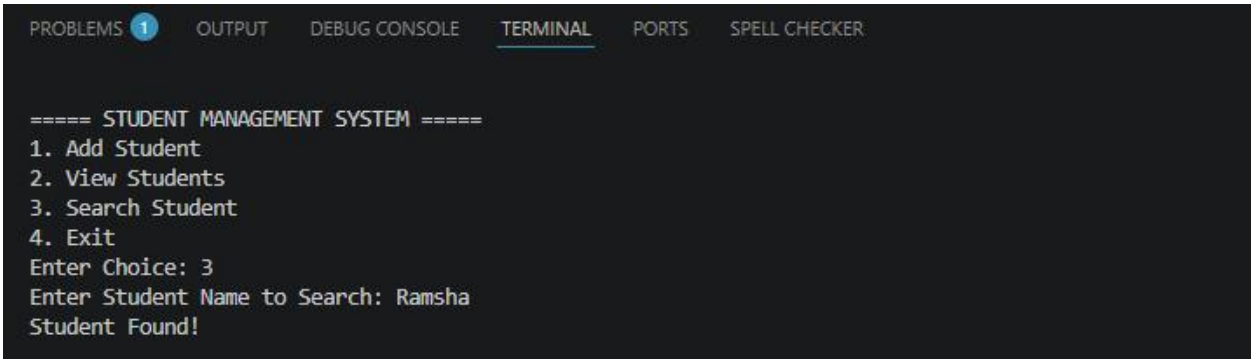
The user selects option 3 (Search Student) and enters the name of the student to be searched.

Output:

The system checks whether the entered student name exists in the stored records and displays an appropriate message indicating whether the student was found or not.

Explanation:

This functionality demonstrates searching operations using the contains() method of the ArrayList. It helps users quickly locate a specific student record.



Screenshots 4: Exiting the Application

Input:

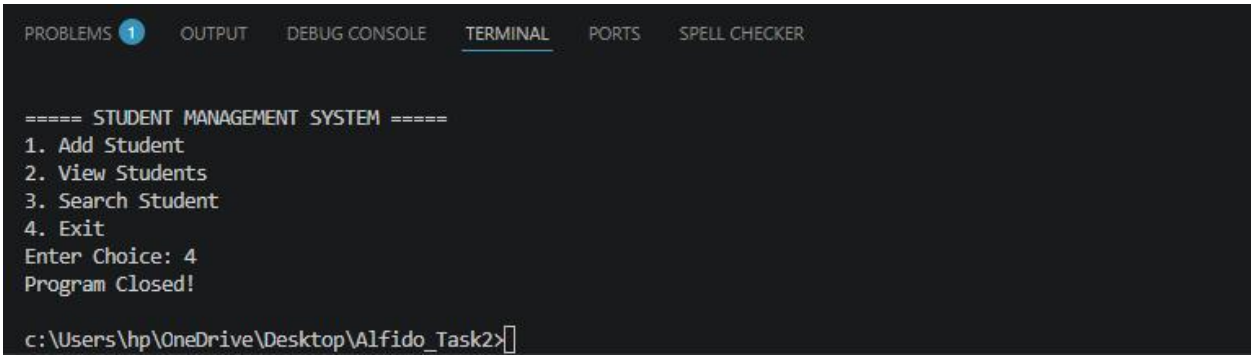
The user selects option 4 (Exit) from the menu.

Output:

The application terminates successfully and displays a closing message.

Explanation:

This functionality demonstrates the use of loop control and conditional statements to safely end the execution of the program when the user chooses to exit.



Concepts Used

1. Scanner Class

The Scanner class is used to take input from the user through the keyboard. It enables the application to interact with users by accepting menu choices and student names. The Scanner class is part of the java.util package and plays an important role in creating interactive Java applications.

Usage in Project:

- Accepting menu choices.
- Taking student names as input.
- Allowing users to search for student records.

2. ArrayList

ArrayList is a dynamic data structure provided by Java that allows elements to be added or removed during program execution. Unlike arrays, ArrayList does not have a fixed size, making it more flexible for managing data.

Usage in Project:

- Storing student names.
- Managing records dynamically.
- Displaying and searching student information.

Benefits:

- Dynamic size.
- Easy insertion and retrieval of data.

- Simplifies data management.

3. Conditional Statements (if-else)

Conditional statements are used to make decisions in a program based on specific conditions. They allow the program to perform different actions depending on the user's input.

Usage in Project:

- Checking the user's menu choice.
- Determining whether a student exists in the system.
- Displaying appropriate messages based on user actions.

Benefits:

- Improves decision-making capability.
- Makes the application interactive and user-friendly.

4. Loops (do-while Loop)

Loops are used to execute a block of code repeatedly until a specified condition becomes false. The do-while loop ensures that the menu is displayed at least once and continues running until the user chooses to exit.

Usage in Project:

- Repeatedly displaying the menu.
- Allowing users to perform multiple operations without restarting the program.

Benefits:

- Reduces code repetition.
- Improves program efficiency.

- Provides continuous interaction with the user**.**

5. String Handling

Strings are used to store and manipulate text data. In this project, strings are used to store student names and compare them during search operations.

Usage in Project:

- Storing student names.
- Searching for specific records.
- Displaying information to the user.

6. Menu-Driven Programming

A menu-driven program provides a list of options from which the user can choose an operation. This approach makes the application simple, organized, and easy to use.

Usage in Project:

- Displaying available operations.
- Allowing users to select desired actions.
- Creating a structured flow of execution.

Advantages:

- User-friendly interface.
- Easy navigation.
- Better organization of program functionalities.

7. Console-Based Application Development

The project is developed as a console-based application where all interactions take place through the terminal. This approach helps beginners understand the fundamentals of programming logic and application flow before moving to graphical user interfaces.

Benefits:

- Easy to develop and test.
- Helps strengthen core programming concepts.
- Builds a foundation for advanced application development.

Learning Outcomes

- Learned how to build a console-based Java application.
- Gained practical experience with Scanner and ArrayList.
- Improved understanding of loops and conditional statements.
- Learned how to store, retrieve, and search data.
- Enhanced problem-solving and logical thinking skills.
- Understood the structure of a menu-driven application.

Conclusion

Through this task, I successfully developed a Student Management System using Java. The project provided practical exposure to important Java concepts such as user input handling, loops, conditional statements, and dynamic data storage using ArrayList.

This task strengthened my programming skills and improved my understanding of how real-world applications are designed and implemented. It also enhanced my confidence in developing Java-based projects independently.

References

- Oracle Java Documentation
- Visual Studio Code Documentation
- Java Programming Learning Resources